Sudoku Solver and Automated Planner

Alex Kempinski
Jamie Scott

Video Demo Link:
https://drive.google.com/file/d/18hOvTrgPmhyZXuabn2aqZLQDlhkmvA7R/view?usp=sharing

Use of Generative AI: GenAI Tools were used to help understand core concepts, such as: heuristics, backtracking, PDDL file structure and syntax. However, GenAI was **NOT** used to generate any of our coding or report writing.

Table Of Contents

Part 1A - Constraint Satisfaction Problem (CSP) **2**
Part 1B - Design and Implement Sudoku Solver **2**
Design Considerations 2
Implementation Process 3
A* Search Analysis 4
Part 2A & 2B- Modelling the Domain and Problems **5**
The Domain File 5
Missions 1 and 2 5
Modelling Decisions 6
Testing 6
Part 2C - Extending the Domain and Problem **7**
The Extended Domain File 7
Mission 3 7
Testing 7
References **8**

Part 1A - Constraint Satisfaction Problem (CSP)

Variables

$X = \{V_{x,y} \mid x \in \{1, 2, \ldots, 9\} \mid y \in \{1, 2, \ldots, 9\}\}$

Domains

$D_i = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

Constraints

1. For the 9 cells in a row, each cell must contain a number 1-9 AND every number must appear only one time
2. For the 9 cells in a column, each cell must contain a number 1-9 AND every number must appear only one time
3. For the 9 grids in a sub-square, each grid must contain a number 1-9 AND every number must appear only one time.

Part 1B - Design and Implement Sudoku Solver

Design Considerations

For our implementation, we decided to implement the sudoku solver using backtracking search as our main search engine along with key Constraint Satisfaction Problem (CSP) heuristic: Minimum Remaining Value (MRV).

We decided on backtracking search as our main search engine as CSPs possess specific structural properties that make specialised depth-first search far more efficient than naive brute-force or uninformed path search.

For our heuristic properties, we chose MRV as it dynamically selects the unassigned cell with the fewest legal values remaining in its domain, directly minimising the search tree's branching factor. From similar studies on solving CSP's, we found MRV is a "commonly employed heuristic" and "tends to focus on the most restricted variables first" which in turn "determines dead ends early" making our algorithm quicker and more efficient as it will effectively negate an amount of invalid paths from being searched needlessly**[1]**.

We believe these CSP techniques will make the implementation more efficient as:

- It removes redundant branching - Variable assignments in a CSP are commutative (A then B equals the same partial state as B then A). Standard backtracking exploits this property by evaluating only one unassigned variable per search depth level, this eliminates redundant search branches.

- MRV prioritises the most restricted variables first, meaning it fails early on invalid paths, which drastically cuts down the search depth.
  .
- Constraint checking - before assigning a value to a cell, the algorithm checks the other values by definition of the constraints 1,2,3. Values that violate said constraints are removed from possible solutions stopping the algorithm from going down invalid paths. And if eventually a value ends up having no valid values, the algorithm recursively backtracks, abandoning this incorrect path.

However, it's worth noting that in pathological worst-case instances, heuristics cannot prevent the asymptotic bound, where the search tree expands exponentially ($O(d^n)$, where d is the number of possible digits in the domain and n is the number of cells). To put this into perspective, if the sudoku board was to be expanded

from a 9x9 grid to a 16x16 grid, the run-time for a no-solution sudoku puzzle would go from 9^{81} to 16^{256}, which is over 910 septdendecillion orders of magnitude bigger for just an extra 175 cells.

Implementation Process

This is the process that we took to implement our final code:

1. Implemented a function that allows the user to enter a sudoku file in the format of .txt or .csv. The program reads the file and converts the file into a 9x9 list of lists, with zeros representing the empty cells.
2. Implemented a function to check the validity of the given sudoku data to make sure it fits the constraints to be correctly translated
3. Implemented a function to check if the given sudoku data is a valid sudoku puzzle by checking there are not any duplicate values in any rows, columns or 3x3 boxes.
4. Implemented a function to translate the given sudoku data, once validated, into a dictionary, representing each cell as its (row, column) coordinates, allowing the solver to access and modify individual values
5. Implemented a function that determines every remaining possible value that can be inserted into an empty cell. Does this by looking through the cells row, column and the 3x3 box it is in, removing any numbers that are present according to the domain
6. Implemented the MRV heuristic to choose the next empty cell, MRV selects the cell with the fewest legal values remaining, helps with reducing the amount of backtracking needed
7. In the same function as the MRV heuristic, we implemented a recursive backtracking algorithm that assigns a legal value to a cell,if the assignment leads to a situation where no valid solution can be found, the algorithm reverses the assignment and assigns another value
8. Implemented a function which displays the solved sudoku puzzle clearly in the 9x9 format, with the 3x3 boxes shown clearly with lines splitting each individual 3x3 box
9. Implemented the main program code which calls all the functions to solve the sudoku puzzle and display the solved puzzle to the user
10. We tested all the given sudoku puzzle files and debugged some issues including validation, incorrect formatting of the output of the solved puzzle, and issues with the conversion into a dictionary, and after debugging we verified the solver can now successfully solve sudoku puzzles of differing scales of difficulty while satisfying the constraints of rows, columns and 3x3 boxes.
11. Implemented a GUI for the sudoku for a user to easily work by loading up a puzzle file, solving a puzzle file, and clearing the puzzle to use again

A* Search Analysis

A* search is a search algorithm that combines the accumulated cost so far of a state with the estimated remaining cost to determine what step it takes next:

f(n) = g(n) + h(n)

f(n) represents the estimated cost of the total search
g(n) represents the cost so far
h(n) represents the estimated cost to reach the end of the search

Using A* search in the sudoku solver:

State Representation

Each state would represent a partially finished 9x9 Sudoku grid. The initial state would represent the given sudoku puzzle to solve, with the empty cells being ‘0’s; The goal state would be the completed and solved sudoku puzzle that satisfies all the given constraints.

Actions
The Algorithm would pick an empty cell and assign it a value in the domain (1,...,9) and that aligns with the given constraints

Path Cost
Each time a cell is assigned a value it could be given for example a path cost of 1, therefore the total path cost would be the number of times an empty cell is assigned a value

Possible heuristic
A heuristic that could be used would be the total number of empty cells left on the sudoku grid;
I.e if a grid had 10 empty cells the heuristic value would be 10.

This heuristic works because every empty cell has to be assigned a value in order for the sudoku to be solved, so the number of actions will always be correct.

Comparing A* with recursive backtracking/MRV
A* search could hypothetically be used to solve sudoku, however it is a much less efficient algorithm compared to the recursive backtracking/MRV algorithm we used in our design, for this specific task.

- Memory - A* uses a priority queue to choose the next cell to be assigned a value, as the number of different configurations of given cells increases, the priority queue will increase drastically causing a large amount of memory storage to be a necessity; whereas the recursive backtracking primarily stores the current assignment cell and can undo assignments without having to add them to the memory
- Efficiency- A* uses its heuristic property to determine and prioritise its search paths next step, but the number of empty cells is slightly pointless way of determining the path with the highest potential as because every valid solution requires the same number of assignments, therefore the heuristic value is going to stay constant regardless of which cell is chosen, so it will default to using the priority queue as its choice for picking the next cell; whereas MRV heuristic determines the empty cells with the fewest legal values, allowing the algorithm to find the cells with the highest chance of being correct on first assignment, which reduces the amount of backtracking needed later on.

Is A* search suited for Sudoku solving?
Although it is possible to do, A* search is not particularly suited to solving sudoku puzzles.

A* search is best used for algorithms where different paths have different costs, whereas the backtracking method directly supports the approach to solving sudoku by exploring possible solutions and backtracking when necessary to assign new values.

Part 2 - Automated Planning (PDDL)
Part 2A & 2B- Modelling the Domain and Problems

The Domain File

Here is an overview of the domain file:

- Object types - rover, lander, waypoint, sample, and supertype data (image or scan)
- Predicates:
    - **rover-lander** - Statically links each rover to its corresponding lander.
    - **rover-at** - Gives the location of the rover, this allows it to move.
    - **data-at / sample-at** - Stores the waypoint of a piece of data/a sample.
    - **connected** - Connects first waypoint to second waypoint, unidirectional.
    - **rover-data / rover-sample** - Records the data/sample a rover is carrying.
    - **transmitted-data** - Records the data that has been transferred to the lander.
    - **lander-sample** - Records the sample a lander is carrying.
    - Other predicates that are mentioned later in the report include: **in-orbit, landed-at, rover-onboard, rover-data-free, rover-sample-free** and **lander-sample-free**.

- Action schemas:
    - **land_lander -** Lands lander at the waypoint given. Precondition checks if the lander is in orbit, meaning it's not yet landed, to make sure landers cannot land more than once.
    - **move_rover** - Moves rover from its current waypoint to the second waypoint given. Precondition checks if the rover is deployed and if the two waypoints are connected in that direction to ensure consistency.
    - **deploy_rover** - Deploys Rover. Precondition checks if the rover is onboard, and the lander has landed.
    - **rover_acquire_data** - Rover acquires a piece of data. Precondition checks the rover is at the same waypoint as the data and it's not currently holding any other pieces of data. Handles data as a supertype, so it can process both images and scans.
    - **rover_transmit_data** - Rover transmits data to the lander. Precondition checks if the rover has data to send and if it's sending it to its corresponding landers, not another lander. Also handles data as a supertype, so it can process both images and scans. Also worth noting it does not check the rovers waypoint, as data can be transmitted.
    - **rover_collect_sample** - Rover collects a sample. Precondition checks if the rover is at the same waypoint as the sample and the rover is not holding any other samples.
    - **rover_store_sample** - Rover stores samples at its lander. Precondition checks if the rovers are linked and at the same waypoint and the lander's sample slot is empty.

Missions 1 and 2

Mission 1 involves a lander and its rover moving through the directed edges in figure 2 of the coursework specification, with the goal of handling two pieces of data and a sample.

Mission 2 involves two landers and their rovers moving through the directed edges in figure 3 of the coursework specification, with the goal of handling four pieces of data and two samples. Rover 1 starts deployed at waypoint 2 and Rover 2 starts aboard Lander 2 (has not yet landed). Arrows going both ways in figure 3 of the coursework specification were treated as pairs and were defined separately using the **connected** predicate.

The follow initial states have to be defined for any Mission using this domain:

- Landers can either begin as in-orbit or landed, using the **in-orbit** and **landed-at** predicates.
- Landers sample slot has to start as free, defined using the **lander-sample-free** predicate.
- Rovers have to be linked to a lander by using the rover-lander predicate.
- Rovers data and sample slots have to start as free, using the **rover-data-free** and **rover-sample-free** predicates.
- Rover can either begin as onboard a lander or deployed, by using the **rover-onboard** or **rover-at** predicates respectively.
- All waypoint connections need to be defined using the **connected** predicate, with the first waypoint being the 'from' waypoint and the second being the 'to' waypoint.
- All locations of images, scans and samples have to be defined using the **data-at** and **sample-at** predicates.

Modelling Decisions

To handle the limited rover and lander memory, the state predicates **rover-data-free**, **lander-sample-free** and **rover-sample-free** represent if the rover/lander's data/sample slot is free or taken. This means they need to be defined in the initial states as free and from then on if it exists, the slot is free and if it doesn't, the slot is taken. As a result of this, landers can only hold one sample at a time and rovers can only hold one piece of data and sample at a time. This is purposely done to avoid using negative pre-conditions, as :strips does not allow negative pre-conditions, so it was encoded as "free", a positive fact.

Also, an assumption we made in our coding solution is that a rover has to be deployed and have a waypoint to be able to handle data and samples. This means the action schemas **rover-acquire-data** and **rover-collect-sample** both check the predicate **rover-at** to see if it has a waypoint, which in turn means its been deployed.

Landing is modelled as an action, **land_lander**, which takes the landing waypoint as a parameter. For missions where a waypoint is not given, the planner chooses the landing waypoint. As the waypoint parameter is unrestricted in the preconditions, it can land at any waypoint. Because landing deletes **in-orbit**, each lander lands at most once and stays at that waypoint.

Testing

To test the domain, Mission 1 and 2 was tested with the domain using BFWS (FF-parser version) via the VSCode PDDL extension.

When running Mission 1 with the domain, the planner finds a 16-step plan in a fraction of a millisecond (0.000269 seconds), expanding only 41 nodes. This shows that the domain models the basic mission actions such as landing, deploying the rover, acquiring/transmitting data and collecting/storing samples compactly. On top of this, the planner can also find a valid plan with almost no search effort when the problem is small.

When running Mission 2 with the domain, the planner finds a 28-step plan in ~1 millisecond (0.001049 seconds), expanding 238 nodes. This shows the robust and flexible nature of our solution and demonstrates it can handle multi-agent co-ordination with minimal search effort, with the plan length growing from 16 to 28 steps as the mission grows.

Part 2C - Extending the Domain and Problem

The Extended Domain File

Here are the additions that were made to the first domain file which resulted in the solution for the extended domain:

- Object Types - astronaut and rooms supertype (controlRoom or dockingBay) were added.
- Predicates added:
  - **astronaut-lander** - Links an astronaut to a lander.
  - **astronaut-at** - Records the room an astronaut is in.
  - **lander-rooms** - Connects a lander to a room.
  - **connect-rooms** - Connects two rooms to allow astronauts to move between them.
- Action Schema added:
  - **move_astronaut** - Moves astronauts from one room to the other. Predicate checks if the astronaut is in the lander, the 'from' room is connected to the 'to' room, both rooms are connected to the lander and the astronaut is in the 'from' room.
- Action Schemas changed:
  - **deploy_rover** - Predicate also now checks if the given astronaut belongs to the lander and is in its docking bay.
  - **rover_transmit_data** - Predicate also now checks if the given astronaut belongs to the lander and is in the control room.
  - **rover_store_sample** - Predicate now checks if the given astronaut belongs to the lander and is in its docking bay.

This means if a rover wants to be deployed or store a sample, an astronaut has to be in the docking bay to do that action, which may result in **move_astronaut** steps. The same applies to the action of a rover transmitting data, where an astronaut has to be in the control room.

Mission 3

Mission 3 involves the same problem space as Mission 2, with the addition of two astronauts, Alice and Bob. They are defined to be on their respective landers, with Alice being in the docking bay and Bob being in the control room.

The following additional initial states have to be defined for any Mission using the extended domain:

- Each room has to be connected to its lander using the **lander-rooms** predicate.
- Every room in the same lander needs to be connected using the **connect-rooms** predicate. This has to be done for every combination as the predicate is unidirectional.
- Each astronaut has to be connected to its lander using the **astronaut-lander** predicate.
- The room every astronaut begins in has to be defined using the **astronaut-at** predicate.

Testing

To test the domain, Mission 3 was tested with the extended domain, also using BFWS (FF-parser version) via the VS Code PDDL extension.

When running Mission 3 with the extended domain, the planner finds a 33-step plan in ~1.4 milliseconds (0.001376 seconds), expanding 367 nodes. With it being 5 steps longer than Mission 2, reflecting the **move_astronaut** actions needed. This shows that the extended domain can handle the added astronaut and room constraints without a significant increase in the search space.

References

[1] Nair, R.R., S, K., Joseph, S., R, D.N. and Sharma, N. (2025) “Constraint Propagation Techniques for Efficient Sudoku Solvers,” *2025 International Conference on Emerging Technologies in Computing and Communication (ETCC)*. IEEE, pp. 1–5. Available at: https://doi.org/10.1109/ETCC65847.2025.11108462.